REVISÃO PARA O ENADE 2026

Sistemas Operacionais

Bacharelado em Ciência da Computação · USCS · Squad 6

Jorge Orselli Mutran · Kaick Kenithi Nishiya · Lucas Koiti Zotovici Kobayashi · Lucas Secanho do Prado · Matheus Pestilli Rodrigues João

Roteiro da revisão

1. **Fundamentos do Sistema Operacional**
   Tópicos 1, 2 e 3

2. **Processos, Sincronização e Escalonamento**
   Tópicos 4 e 5

3. **Gerência de Memória Principal**
   Tópico 6

4. **Memória Virtual**
   Tópico 7

5. **Sistemas de Arquivos e Entrada/Saída**
   Tópicos 8 e 9

Seções 1, 2 e 5 agrupam mais de um tópico da matriz; as demais correspondem a um tópico cada.

1

Fundamentos do Sistema Operacional

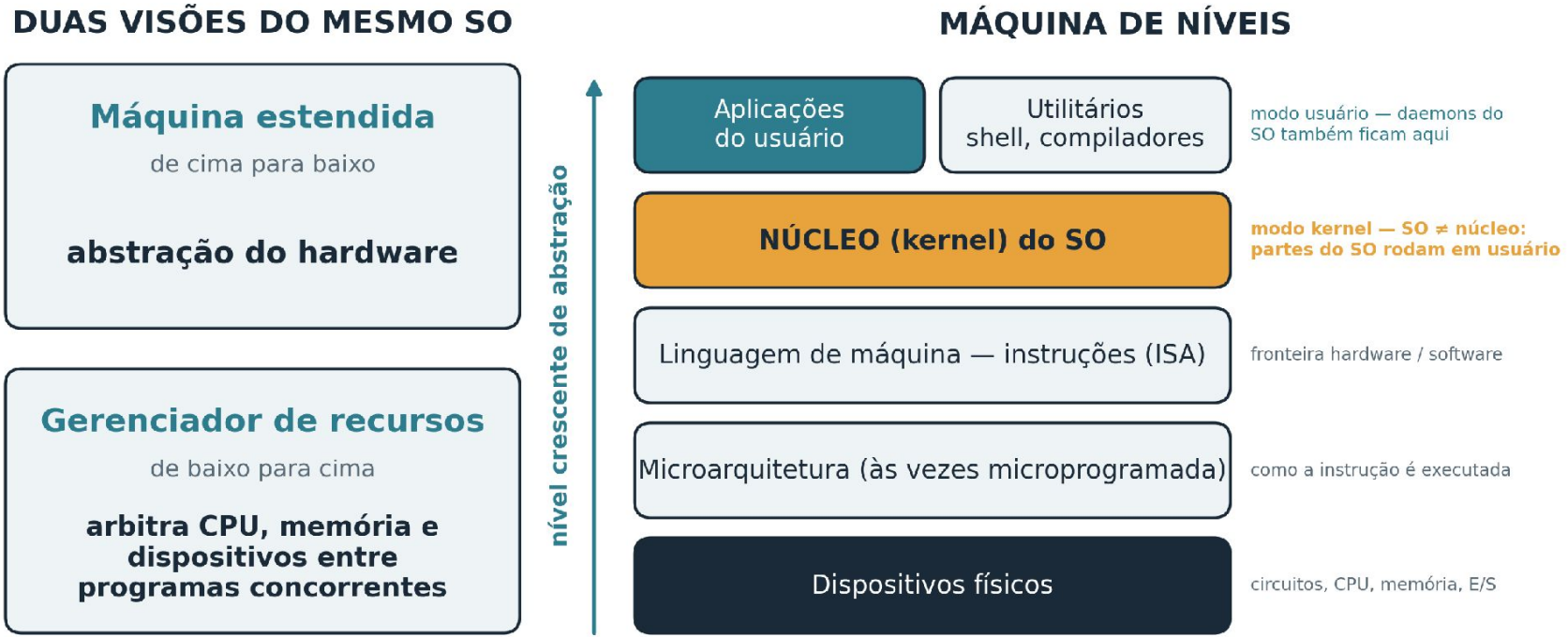
Tópicos 1, 2 e 3 — visão geral, modos de acesso e arquitetura do núcleo

T1 · Visão geral, máquina de níveis e softwares tradutores

T2 · Modo usuário/kernel, interrupções, exceções e chamadas de sistema

T3 · Arquiteturas de núcleo, boot e virtualização

Duas visões do SO e a máquina de níveis



Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Como os SO são classificados

EIXO 1 Programas e usuários	EIXO 2 Modo de processamento	EIXO 3 Múltiplos processadores
monoprogramação × multiprogramação	em lote (batch)	um processador × multiprocessador
multitarefa cooperativa × preemptiva	tempo compartilhado (interativo)	fortemente acoplados: memória compartilhada, um só SO (SMP × AMP)
monousuário × multiusuário	tempo real: crítico (hard) × brando (soft)	fracamente acoplados: cada nó com seu SO (SO de rede × distribuído)

É a classificação de Machado & Maia — a que a Matriz de Referência usa. Tempo compartilhado e tempo real pressupõem multiprogramação (o lote da 2ª geração era monoprogramado).

Modo usuário × modo kernel

A diferença em uma linha — no modo KERNEL a CPU executa qualquer instrução e acessa toda a memória e todos os dispositivos; no modo USUÁRIO ela só executa o subconjunto não privilegiado e só acessa a memória do próprio processo.

Bit de modo — um bit no registrador de status do processador diz em qual modo a CPU está executando.

Instruções privilegiadas — desabilitar interrupções, acessar registradores de controladores, alterar a tabela de páginas. Só executam em modo kernel.

Tentou em modo usuário? — o HARDWARE gera uma exceção e transfere o controle ao núcleo, que normalmente encerra o processo.

Por que importa — sem isso, qualquer programa poderia derrubar o sistema ou ler a memória dos outros.

Tópico 2 · três formas de entrar no núcleo — não confunda

Interrupção, exceção e chamada de sistema

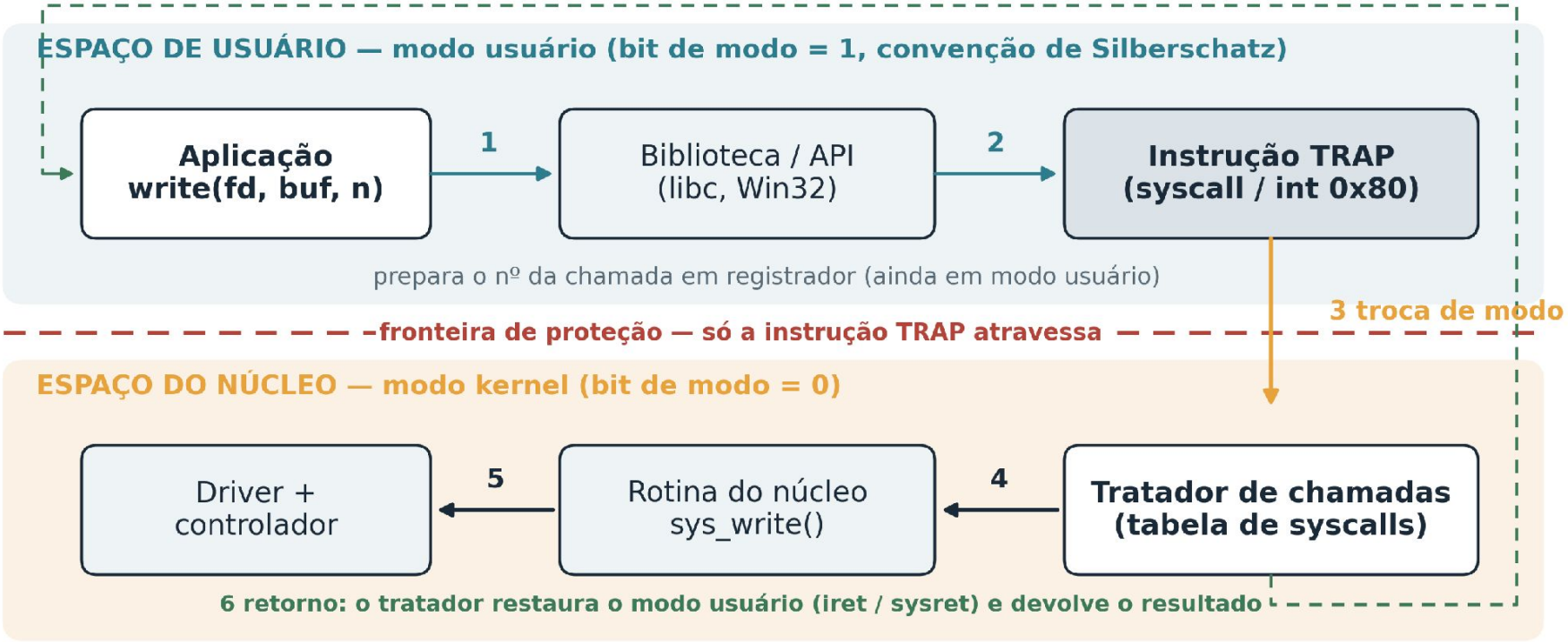
Interrupção — ASSÍNCRONA. Vem de um dispositivo EXTERNO: disco terminou, temporizador expirou, tecla foi pressionada. Não tem relação com a instrução em curso.

Exceção (trap) — SÍNCRONA e involuntária. Causada pela própria instrução: divisão por zero, endereço inválido, instrução privilegiada em modo usuário, falta de página.

Chamada de sistema — SÍNCRONA e VOLUNTÁRIA. O processo pede um serviço ao núcleo executando uma instrução de trap.

API ≠ system call — a biblioteca (libc, Win32) roda em modo usuário. Há funções que não geram nenhuma syscall e outras que geram várias.

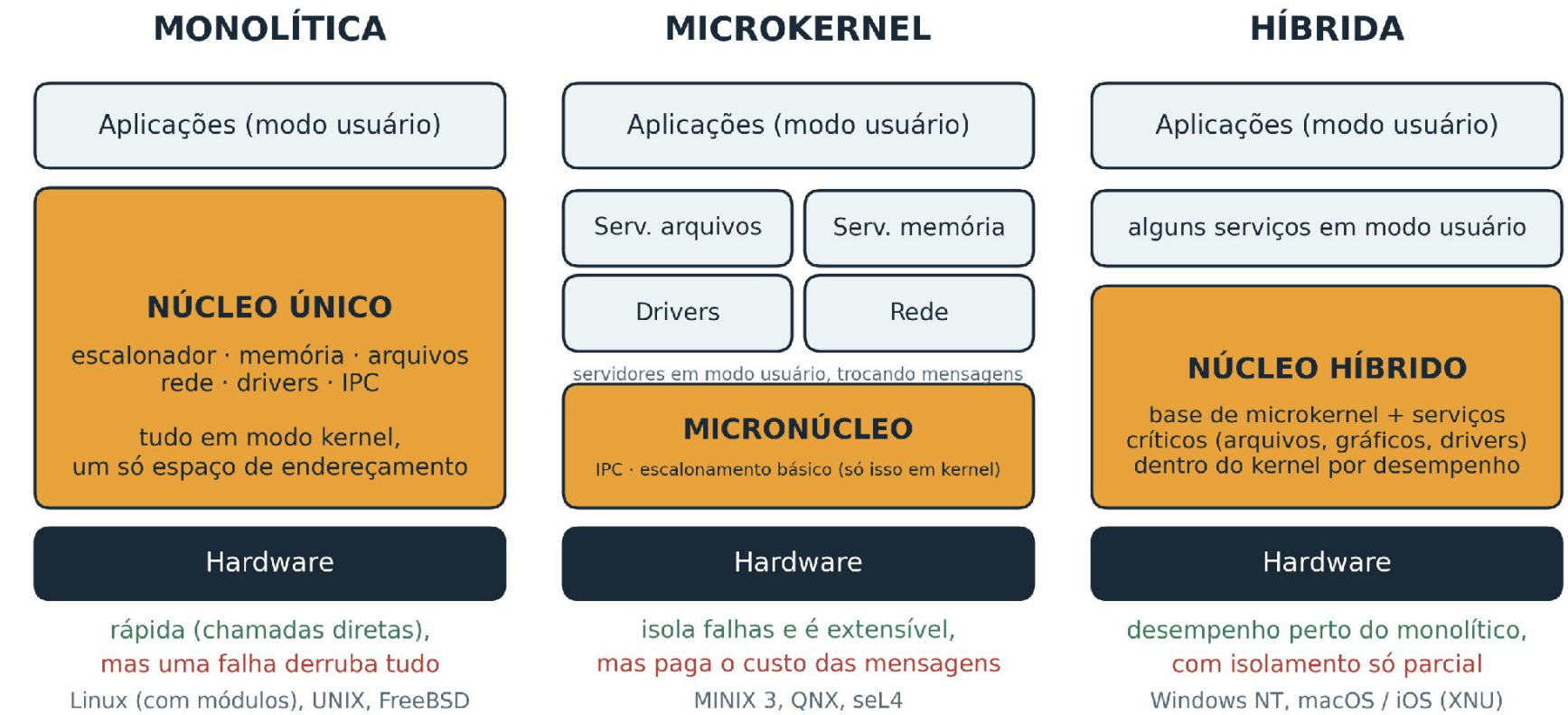
O caminho completo de uma chamada de sistema



printf() e fputs() têm buffer: podem gerar 0, 1 ou várias chamadas write() — a função de biblioteca NÃO é a syscall

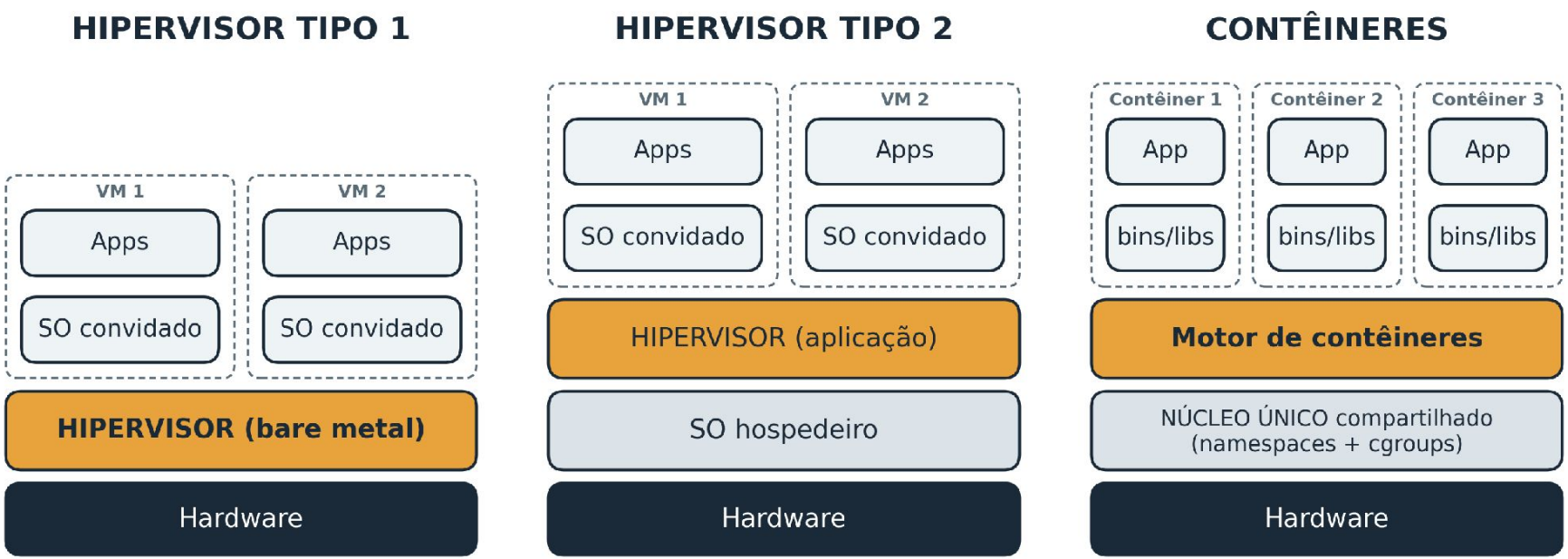
Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Arquiteturas de núcleo



Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Virtualização e contêineres



** KVM: Silberschatz (10ª ed.) o classifica como tipo 1 (SO de propósito geral com funções de VMM); parte da literatura o trata como tipo 2 por depender do Linux hospedeiro.*

Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

QUESTÃO 1 · ENADE

Questão de Alternativas Constantes — V/F (CAT 2)

SITUAÇÃO-PROBLEMA

Uma aplicação de terceiros tentou escrever diretamente no registrador de controle da controladora de disco e foi encerrada pelo sistema operacional com erro de violação de acesso. A equipe revisita então os mecanismos de proteção por modo de acesso.

COMANDO

Analise as afirmações a seguir sobre modo de acesso e transferência de controle ao núcleo, identificando-as como Verdadeiras (V) ou Falsas (F).

( ) A instrução que desabilita interrupções é privilegiada; se um processo em modo usuário tentar executá-la, o hardware gera uma exceção e passa o controle ao núcleo.
( ) A interrupção é um evento síncrono, provocado pela instrução em execução, como uma divisão por zero ou um acesso a endereço inválido.
( ) A chamada de sistema é o mecanismo pelo qual um processo pede um serviço ao núcleo, trocando de modo usuário para modo kernel por meio de um trap.
( ) A API é sinônimo de chamada de sistema, pois toda função de biblioteca corresponde a uma única syscall executada em modo kernel.

Assinale a alternativa com a sequência correta.

ALTERNATIVAS

A) V – F – V – F

B) V – V – F – F

C) F – V – V – V

D) V – F – F – V

QUESTÃO 1 · GABARITO: A

A (V) Instruções privilegiadas só executam em modo kernel; a tentativa em modo usuário dispara uma exceção tratada pelo SO — é exatamente o que ocorreu no incidente descrito.

B Considera verdadeira a segunda afirmação, trocando exceção por interrupção — o erro de terminologia mais frequente do tópico. E rejeita a terceira, que descreve corretamente o funcionamento da chamada de sistema.

C Nega a primeira afirmação, que está correta e é o próprio mecanismo de proteção que explica o incidente. Além disso, aceita a quarta, confundindo API com chamada de sistema.

D Acerta as duas primeiras avaliações, mas inverte as duas últimas. Nega a definição correta de chamada de sistema e aceita a equivalência entre API e system call.

2

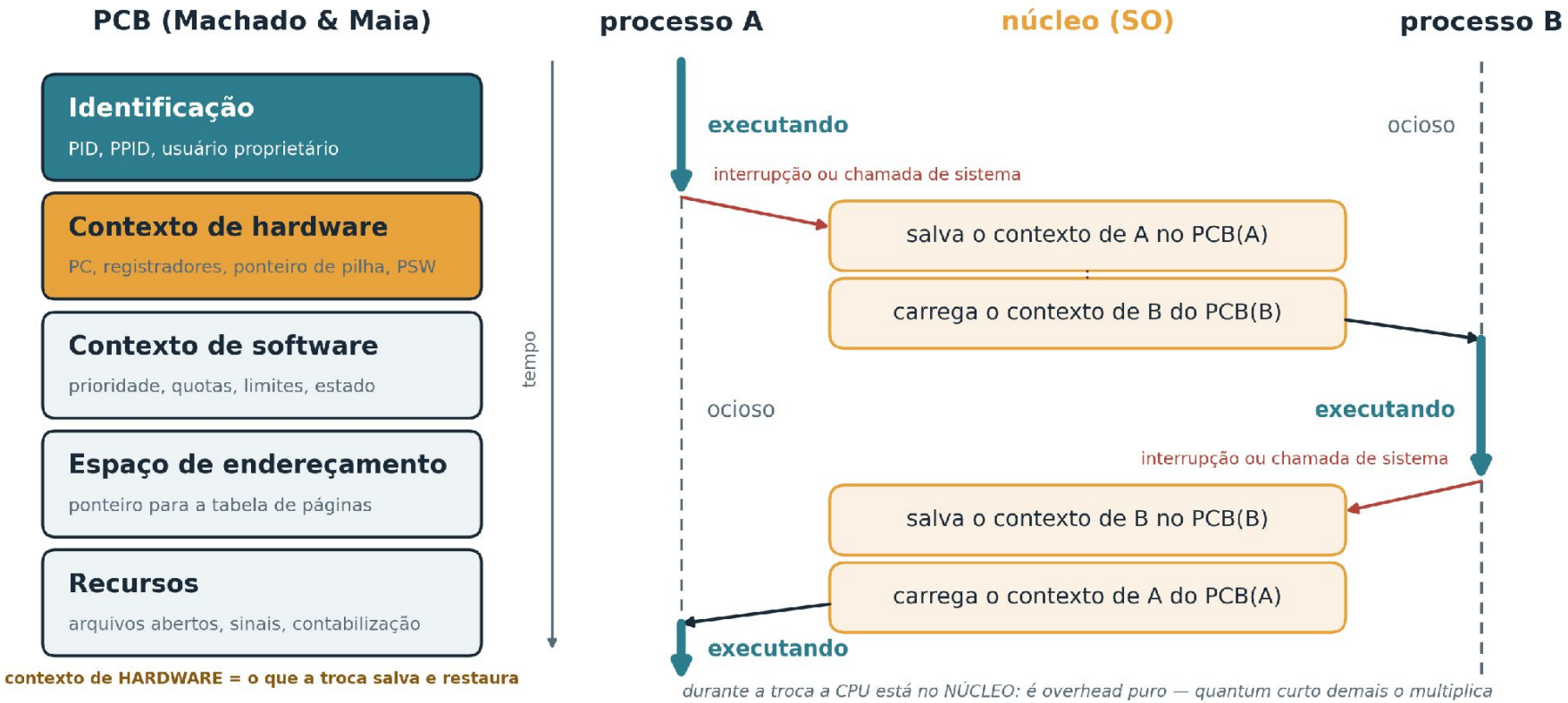
Processos, Sincronização e Escalonamento

Tópicos 4 e 5 — PCB, estados, concorrência, deadlock e uso da CPU

T4 · Processos e threads: PCB, estados, concorrência, sincronização e deadlock

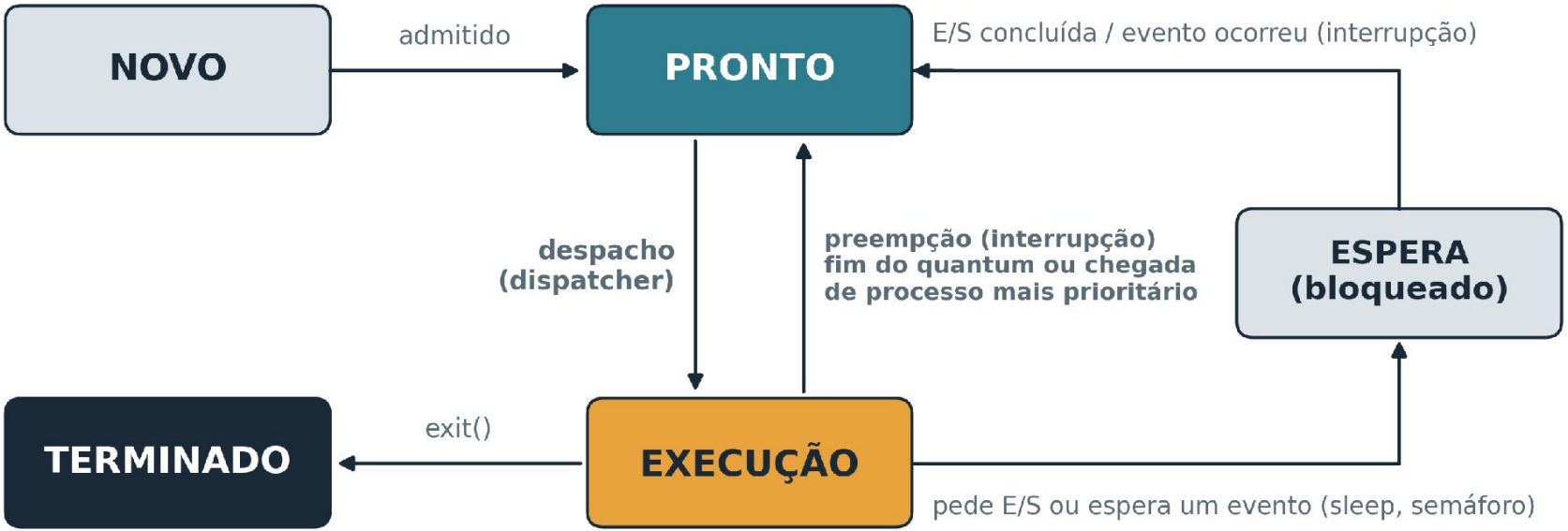
T5 · Compartilhamento do processador e algoritmos de escalonamento

Processo e o Bloco de Controle de Processo



Elaboração dos autores, com base em Machado e Maia (2013) e Silberschatz et al. (2018).

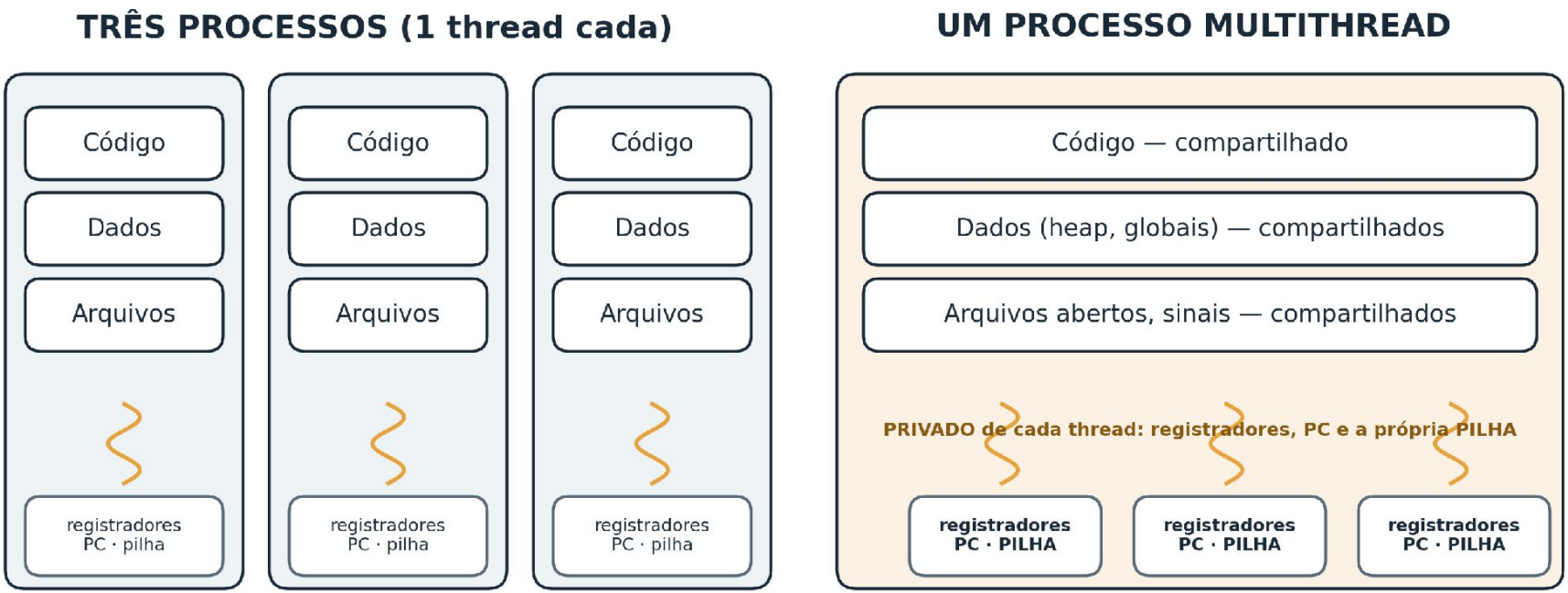
Estados e transições do processo



só a transição PRONTO → EXECUÇÃO é decisão do escalonador; as demais são consequência de eventos

Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Processo × thread



threads do MESMO processo compartilham memória; de processos diferentes, não · uma thread não migra de processo · multithread roda até em 1 CPU (concorrência ≠ paralelismo)

Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Threads de usuário × threads de kernel

Threads de usuário (modo usuário) — criadas e escalonadas por uma biblioteca; o kernel enxerga UM só fluxo. Troca de contexto barata, mas se uma thread bloqueia em E/S o processo inteiro bloqueia, e não há paralelismo real em multicore.

Threads de kernel — o núcleo conhece e escalona cada thread: uma bloqueada não trava as outras e várias rodam em paralelo nos núcleos. Troca mais cara (passa pelo kernel). É o modelo do Linux e do Windows.

Modelos de mapeamento — muitos-para-um (só usuário) · um-para-um (Linux, Windows) · muitos-para-muitos (M:N, Solaris antigo).

O que a prova pergunta (2008, 2011, 2014) — threads do mesmo processo compartilham código, dados e arquivos; cada uma tem PC, registradores e PILHA próprios; não migram entre processos; multithread roda em 1 núcleo (concorrência) e só ganha paralelismo com vários núcleos.

Concorrência: condição de corrida e região crítica

Condição de corrida — o resultado depende da ordem em que os fluxos são escalonados. O mesmo código produz resultados diferentes a cada execução.

Região crítica — o trecho que acessa o recurso compartilhado.

As 4 condições de uma boa solução — 1) nunca dois na região crítica ao mesmo tempo; 2) nada suposto sobre velocidade ou nº de CPUs; 3) quem está fora não bloqueia ninguém; 4) ninguém espera para sempre.

Mecanismos — semáforo (contador + fila; bloqueia) · mutex (binário, com dono) · monitor (a linguagem faz o lock) · Peterson (só software) · TSL/test-and-set (instrução atômica; spinlock) · desabilitar interrupções (só o kernel, só em 1 CPU).

Sincronização na prática

Desabilitar interrupções — impede a preempção enquanto a CPU está na região crítica. Só o kernel pode fazê-lo, e só funciona em MONOPROCESSADOR: outra CPU continua entrando. Foi a asserção-razão do ENADE 2021.

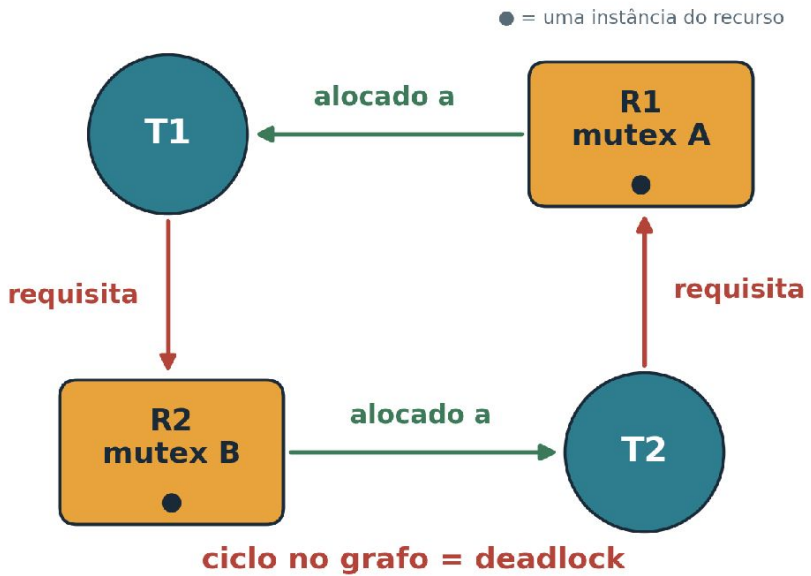
TSL / test-and-set e spinlock — instrução atômica de hardware que lê e escreve a variável de trava sem poder ser interrompida; a espera é OCUPADA (spin) — gasta CPU, mas é rápida para trechos curtos e funciona em multiprocessador.

Semáforo bloqueia; spinlock gira — down() sem crédito coloca o processo em ESPERA (sem gastar CPU); mutex = semáforo binário com dono; monitor = a linguagem faz o lock por você (synchronized em Java).

Produtor–consumidor (ENADE 2005 e 2014) — mutex = 1, vazio = N, cheio = 0. Produtor: down(vazio), down(mutex), insere, up(mutex), up(cheio). Consumidor: down(cheio), down(mutex), retira, up(mutex), up(vazio). Inverter down(mutex) com down(vazio) causa deadlock.

Deadlock: as quatro condições de Coffman

GRAFO DE ALOCAÇÃO DE RECURSOS



ciclo no grafo = deadlock

vale quando cada recurso tem UMA instância

com várias instâncias, o ciclo é necessário mas NÃO suficiente

AS 4 CONDIÇÕES DE COFFMAN

1. **Exclusão mútua**
   só um processo por vez pode usar o recurso
2. **Posse e espera**
   quem detém um recurso pode pedir outro sem liberar
3. **Não preempção**
   não se retira o recurso à força de quem o detém
4. **Espera circular**
   há uma cadeia fechada de espera entre os processos

as quatro precisam valer ao mesmo tempo — quebrar uma delas previne o impasse

Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Tratamento de deadlock: quatro estratégias

Avestruz (ignorar) — assume que é raro e que conviver sai mais barato que prevenir; é o que Linux e Windows fazem para a maioria dos recursos.

Prevenção — projetar o sistema para negar UMA das quatro condições. A mais usada: ordem global de aquisição (todos pegam A antes de B) quebra a espera circular — foi a resposta do ENADE 2017.

Evitar (avoidance) — decidir a cada pedido se o sistema continua em ESTADO SEGURO — algoritmo do banqueiro; exige saber a necessidade máxima de cada processo, o que é raro na prática.

Detecção e recuperação — deixar acontecer, detectar (ciclo no grafo ou matriz de alocação) e recuperar: abortar um processo, preemptar um recurso ou voltar a um ponto de verificação (rollback).

Critérios: o que se quer otimizar

Utilização de CPU — manter o processador ocupado — maximizar.

Throughput (vazão) — processos concluídos por unidade de tempo — maximizar.

Turnaround — conclusão menos chegada — o tempo total do ponto de vista do usuário — minimizar.

Tempo de espera — quanto tempo o processo passou na fila de PRONTO — minimizar.

Tempo de resposta — da submissão até a PRIMEIRA resposta. É o critério dos sistemas interativos — minimizar.

Os algoritmos de escalonamento

FIFO / FCFS — não preemptivo. Simples e justo na ordem, mas um processo longo atrasa todos (efeito comboio). Sistemas em lote.

SJF e SRTF — menor tarefa primeiro; SRTF é a versão preemptiva. Menor tempo médio de espera — para chegadas simultâneas e sem preempção —, mas exige prever a duração e causa inanição dos longos.

Round-Robin — preemptivo por quantum. Limita o tempo de resposta — é a escolha do tempo compartilhado. Quantum pequeno demais = excesso de troca de contexto; grande demais = vira FCFS.

Por prioridade e múltiplas filas — atende classes distintas; prioridade fixa causa inanição → envelhecimento (aging). Múltiplas filas COM realimentação movem o processo de fila conforme o comportamento.

Tempo real — taxas monotônicas (RMS: prioridade fixa, quanto menor o período maior a prioridade) e prazo mais curto primeiro (EDF: prioridade dinâmica).

Qual algoritmo para qual ambiente (Tanenbaum)

Sistemas em LOTE — ninguém espera no terminal: maximizar vazão e uso da CPU, sem preempção → FCFS, SJF (menor job primeiro) e SRTN.

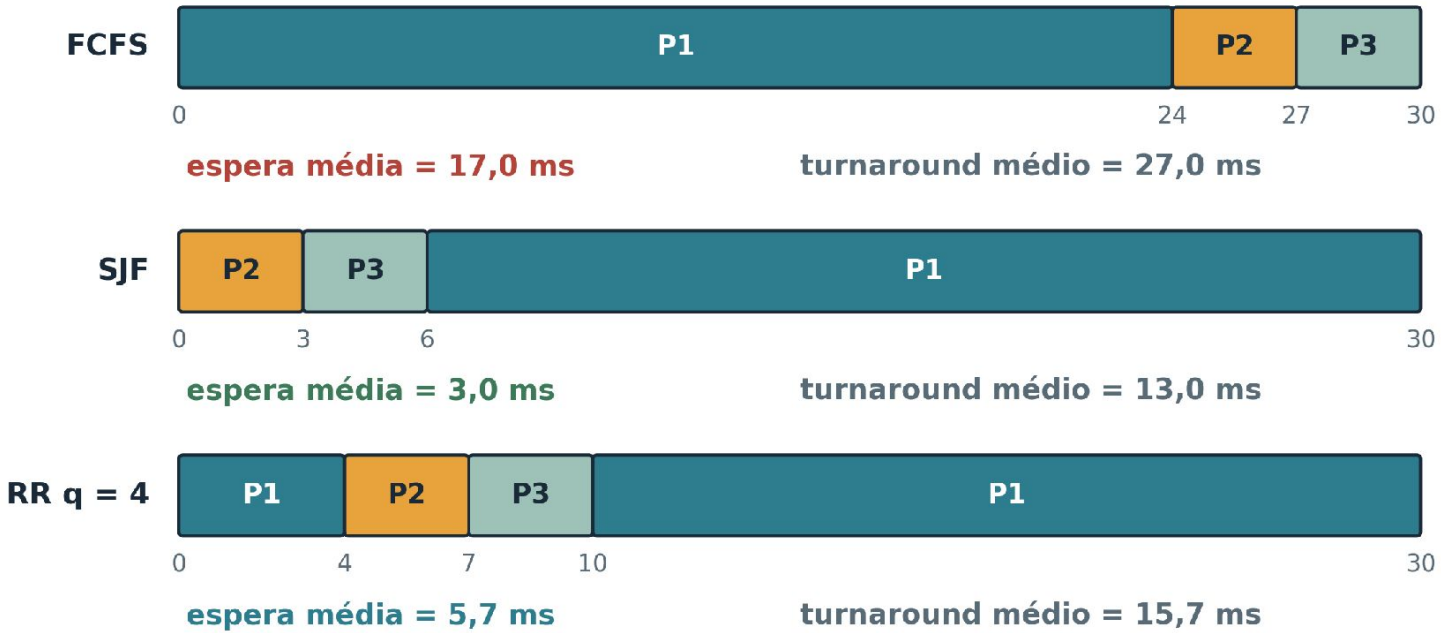
Sistemas INTERATIVOS — usuário esperando: minimizar tempo de resposta, preempção obrigatória → Round-Robin, prioridades, múltiplas filas, loteria e fair-share.

Sistemas de TEMPO REAL — cumprir prazos e ser previsível → RMS (taxa monotônica, estático) e EDF (prazo mais curto primeiro, dinâmico); inversão de prioridade se resolve com herança de prioridade (ENADE 2008).

Objetivos comuns a todos — justiça, respeito à política e equilíbrio (CPU e E/S ocupadas). Os distratores de 2021 eram 'FILO para tempo real', 'RMS para lote', 'SJF para interativo' e 'RR para tempo real'.

O mesmo conjunto, três resultados

P1 = 24 ms · P2 = 3 ms · P3 = 3 ms — todos chegam em t = 0 (exemplo de Silberschatz)



espera = tempo na fila de PRONTO · turnaround = conclusão − chegada = espera + duração · com chegadas diferentes, subtraia a chegada

Elaboração dos autores; exemplo numérico de Silberschatz et al. (2018).

QUESTÃO 2 · ENADE

Questão de Asserção-Razão (CAT 3)

SITUAÇÃO-PROBLEMA

Um sistema de reservas protege a agenda de salas com o mutex A e o cadastro de participantes com o mutex B. A rotina de criação adquire A e, ainda de posse dele, pede B; a de cancelamento adquire B e pede A. Todas as regiões críticas estão corretamente delimitadas. Em produção, sob carga, as duas rotinas travam indefinidamente.

COMANDO

Avalie as asserções a seguir e a relação proposta entre elas.

I. O uso correto de semáforos binários (mutexes) em torno de todas as regiões críticas do sistema descrito é suficiente para garantir que ele não entre em deadlock,

PORQUE

II. a exclusão mútua assegurada pelos semáforos impede que dois fluxos acessem simultaneamente a mesma estrutura compartilhada, eliminando as condições de corrida.

A respeito dessas asserções, assinale a opção correta.

ALTERNATIVAS

A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
E) As asserções I e II são proposições falsas.

QUESTÃO 2 · GABARITO: D

A Exigiria que a asserção I fosse verdadeira, o que o próprio cenário desmente: o sistema usa mutexes corretamente e mesmo assim trava.

B Também parte do pressuposto de que I é verdadeira. Marca quem percebeu que exclusão mútua e deadlock são fenômenos distintos, mas não chegou a concluir que a primeira asserção é, ela própria, falsa.

C Inverte por completo os valores lógicos. A asserção II é uma definição correta de exclusão mútua e do papel dos semáforos; negá-la é desconhecer a função primária do mecanismo.

D A asserção I é falsa: exclusão mútua não previne impasse — ao contrário, ela é a primeira das quatro condições de Coffman.

E Nega também a asserção II, que está correta. Atrai quem, ao identificar o erro da primeira asserção, generaliza a desconfiança e passa a duvidar da utilidade do próprio semáforo.

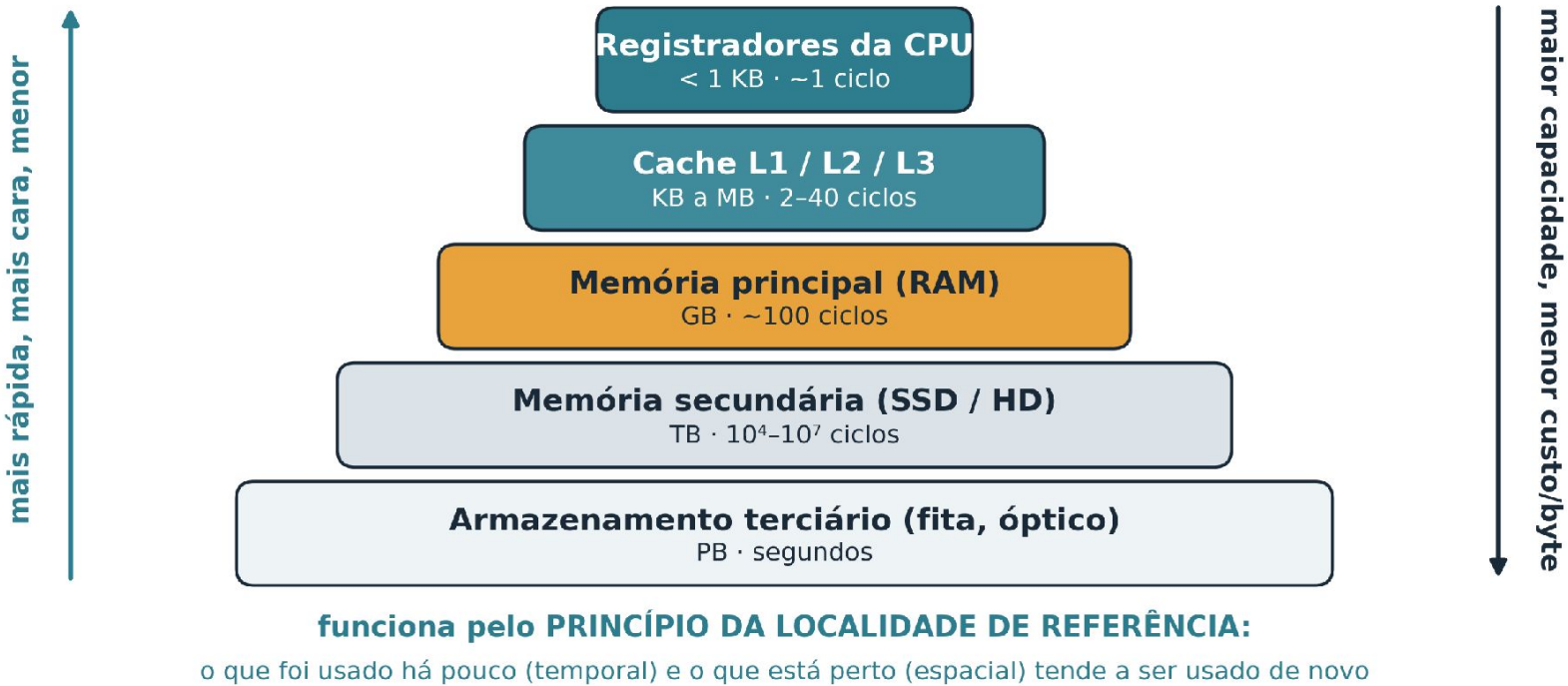
3

Gerência de Memória Principal

Tópico 6 — hierarquia, particionamento, fragmentação e relocação

T6 · Memória principal: alocação contígua, particionamento, endereçamento e relocação

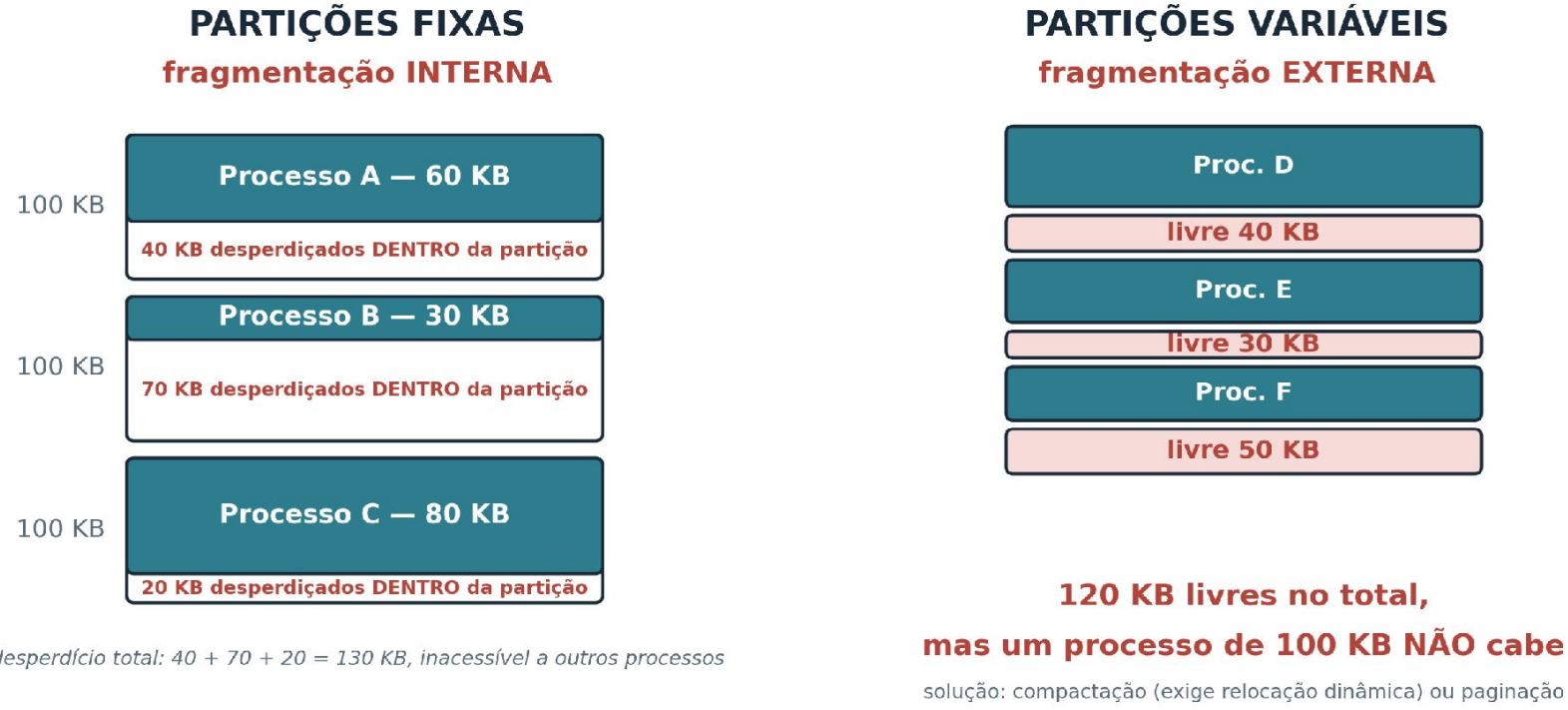
Hierarquia de memória



Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Tópico 6 · a distinção mais cobrada do tópico

Particionamento e os dois tipos de fragmentação



Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Estratégias de alocação

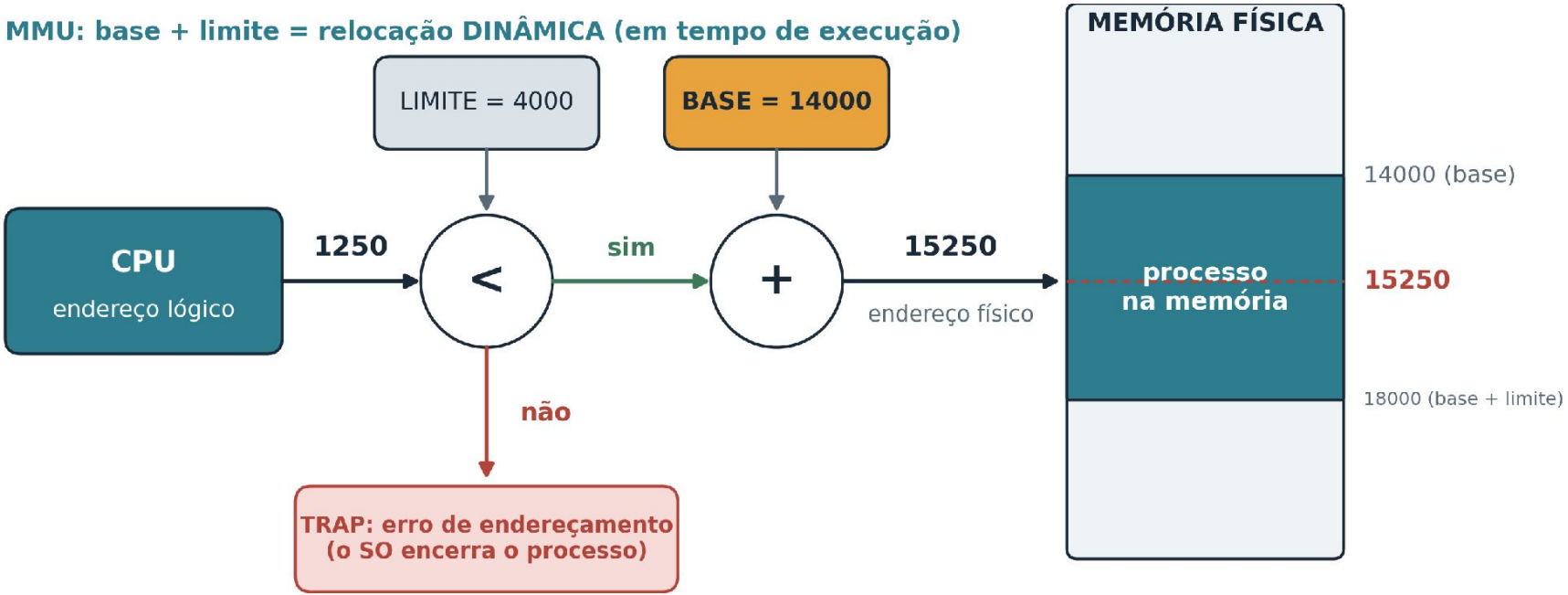
First-fit — a primeira lacuna que couber. É a mais rápida, porque para de procurar assim que encontra.

Best-fit — a MENOR lacuna que couber. Deixa o menor resto, mas exige varrer a lista inteira e espalha lacunas pequenas demais para reusar.

Worst-fit — a MAIOR lacuna de todas, com o argumento de deixar um resto grande e reaproveitável. Na prática, desempenho ruim.

Compactação — junta as áreas livres num único bloco contíguo. Resolve a fragmentação externa, mas custa caro e exige relocação dinâmica.

Endereço lógico, endereço físico e relocação



Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

QUESTÃO 3 · ENADE

Questão de Resposta Única (CAT 1)

SITUAÇÃO-PROBLEMA

Um sistema embarcado usa partições variáveis com alocação contígua. O mapa de áreas livres apresenta, em ordem crescente de endereço, blocos de 100 KB, 500 KB, 200 KB, 300 KB e 600 KB. Chega a requisição de um processo que precisa de 212 KB contíguos.

COMANDO

Considerando o funcionamento das estratégias de alocação e os tipos de fragmentação, assinale a alternativa que indica corretamente a área escolhida pela estratégia best-fit e o tipo de fragmentação característico desse esquema de gerenciamento.

ALTERNATIVAS

A) A área de 500 KB; a fragmentação característica é a interna, pois sobra espaço não utilizado dentro da partição alocada ao processo.

B) A área de 300 KB; a fragmentação característica é a externa, pois restam lacunas livres pequenas e dispersas, incapazes de atender a novos pedidos.

C) A área de 300 KB; a fragmentação característica é a interna, pois a partição tem tamanho predefinido e sempre maior que o do processo.

D) A área de 600 KB; a fragmentação característica é a externa, pois a maior lacuna disponível é subdividida a cada nova alocação realizada.

A A área de 500 KB é a resposta do FIRST-FIT, que aceita a primeira lacuna suficientemente grande sem examinar as demais.

B O best-fit percorre todas as áreas livres e escolhe a MENOR que ainda comporta o pedido: entre 500 KB, 300 KB e 600 KB, a menor suficiente para 212 KB é a de 300 KB, deixando um resto de 88 KB.

C Acerta a área do best-fit, mas erra a fragmentação. Em partições variáveis a partição nasce com o tamanho exato do processo — não existe 'tamanho predefinido maior'; o desperdício fica FORA das partições.

D A área de 600 KB é a resposta do WORST-FIT, que escolhe sempre a maior lacuna com o argumento de deixar um resto grande e reaproveitável.

Questão 3 · resolução passo a passo

First-fit, best-fit e worst-fit sobre o mesmo mapa

Áreas livres na memória, em ordem crescente de endereço (blocos ocupados entre elas)



chega um processo que requisita 212 KB

first-fit	a PRIMEIRA que couber, varrendo do início	**→ escolhe a área de 500 KB (resto 288 KB)**
best-fit	a MENOR que couber (varre a lista toda)	**→ escolhe a área de 300 KB (resto 88 KB)**
worst-fit	a MAIOR de todas	**→ escolhe a área de 600 KB (resto 388 KB)**

best-fit deixa o menor resto (88 KB), mas espalha lacunas pequenas demais para reusar;

first-fit é o mais rápido (para na primeira); worst-fit tem o pior desempenho nas simulações

o mesmo mapa é o exercício clássico de Silberschatz (100, 500, 200, 300, 600 KB) — treinem também com pedidos de 417, 112 e 426 KB

Elaboração dos autores.

4

Memória Virtual

Tópico 7 — paginação, segmentação, TLB, substituição e thrashing

T7 · Memória virtual: paginação, segmentação, mapeamento e substituição de páginas

Memória virtual, paginação e segmentação

Memória virtual — o espaço lógico é MAIOR que o físico: só as partes em uso ficam na RAM, o resto espera no disco; some a exigência de contiguidade (os quadros podem estar em qualquer lugar). Preço: tradução em todo acesso e faltas de página milhões de vezes mais lentas que a RAM.

Paginação — blocos de TAMANHO FIXO, divisão física. Gera fragmentação INTERNA (só na última página) e nenhuma externa.

Segmentação — blocos de TAMANHO VARIÁVEL com significado lógico (código, pilha, dados). Facilita proteção e compartilhamento, mas reintroduz a fragmentação EXTERNA.

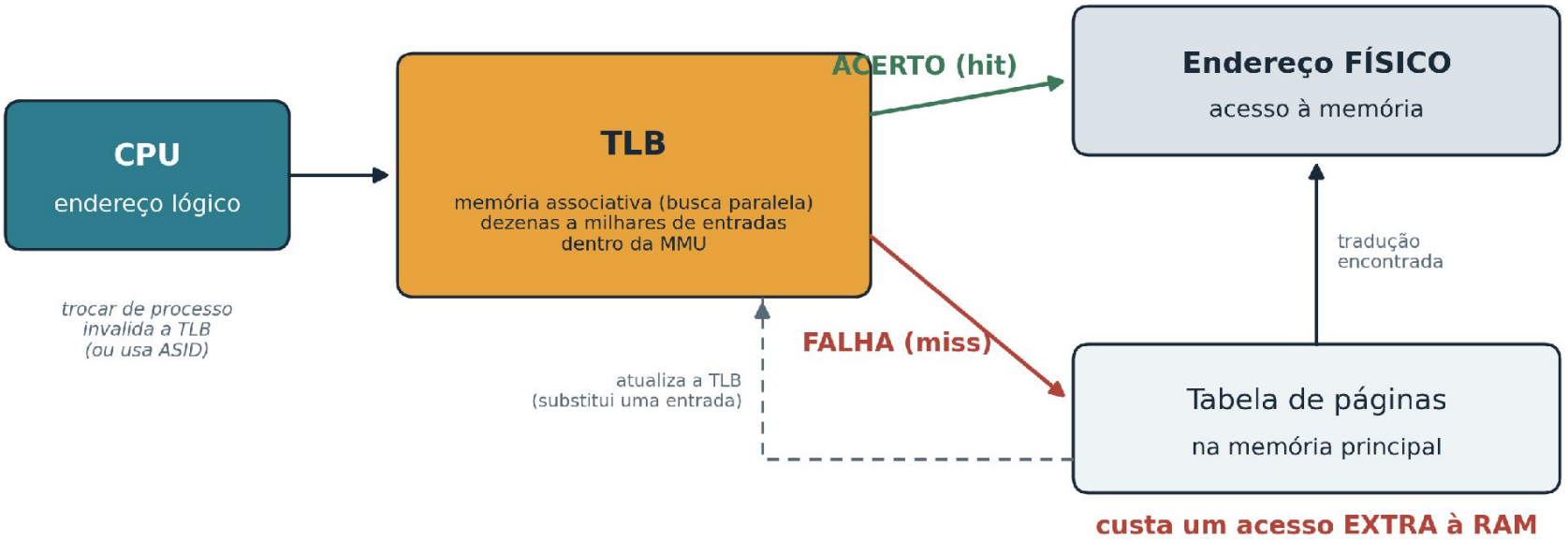
Segmentação paginada — segmentos divididos em páginas: proteção lógica sem fragmentação externa. Foi o modelo do IA-32; em x86-64 a segmentação é vestigial e prevalece a paginação multinível.

Paginação: como o endereço é traduzido



Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

TLB: sem ela cada acesso à memória custaria o dobro



Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Falta de página e políticas de substituição

Sequência da falta — trap → o SO confere se a referência é válida (senão, encerra o processo) → há quadro livre? senão escolhe uma vítima → se ela está modificada (bit M), grava no disco → carrega a página → atualiza a tabela → reexecuta a instrução.

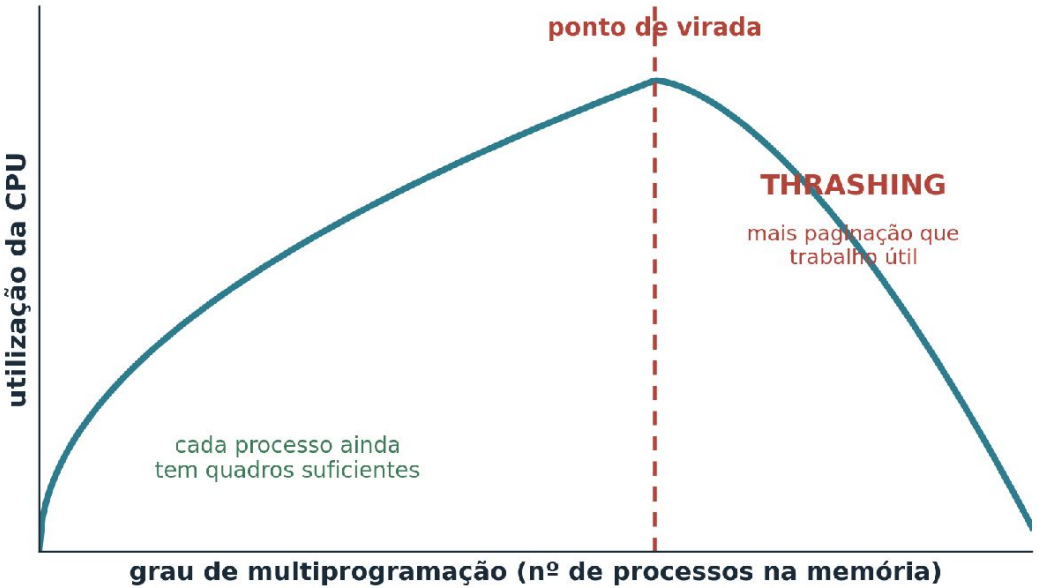
FIFO — substitui a mais antiga. Simples, mas sofre a anomalia de Belady: mais quadros podem dar MAIS faltas — cadeia 1-2-3-4-1-2-5-1-2-3-4-5: 3 quadros = 9 faltas, 4 quadros = 10 (foi a questão do ENADE 2017).

LRU — substitui a menos recentemente usada. Aproxima bem o ótimo, mas exige apoio de hardware (contador ou pilha atualizados a cada acesso).

Ótimo (OPT) — substitui a que será usada mais tarde (ou nunca). Exige conhecer o futuro: serve como REFERÊNCIA teórica, não é implementável.

NRU, segunda chance e clock — aproximações práticas: NRU classifica pelos bits R e M; segunda chance/clock é o FIFO que poupa a página com R = 1 (zera o bit e dá outra volta).

Working set, thrashing e alocação de quadros



Working set W(t, Δ)

páginas referenciadas nas últimas Δ referências (Denning, 1968) — o que o processo realmente precisa AGORA

Thrashing

Σ working sets > quadros disponíveis: falta de página em cascata, disco saturado, CPU ociosa — e o escalonador de longo prazo admite MAIS processos

Remédio

REDUZIR o grau de multiprogramação (suspender processos) · alocar quadros pelo working set ou pela frequência de faltas (PFF) · política LOCAL isola um processo dos demais; a GLOBAL não

alocação de quadros: fixa (igual) ou proporcional ao tamanho / prioridade

Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

QUESTÃO 4 · ENADE

Questão de Resposta Única com cálculo aplicado (CAT 1)

SITUAÇÃO-PROBLEMA

A arquiteta de um sistema operacional para dispositivos de borda precisa escolher a política de substituição de páginas. Para fundamentar a decisão, simula três algoritmos sobre a mesma carga, com três quadros e memória inicialmente vazia.

DADOS DE SUPORTE

Cadeia de referências a páginas utilizada na simulação:

7 · 0 · 1 · 2 · 0 · 3 · 0 · 4 · 2 · 3

Número de quadros disponíveis: 3. Memória inicialmente vazia (todas as faltas iniciais devem ser contabilizadas).

COMANDO

Ao simular os algoritmos FIFO, LRU e Ótimo sobre a cadeia de referências apresentada, qual é o número de faltas de página produzido por cada um deles, respectivamente, e qual conclusão de projeto se sustenta a partir do resultado?

ALTERNATIVAS

A) 9, 8 e 6 faltas; o algoritmo Ótimo serve apenas como referência teórica de desempenho, e o LRU é a política implementável mais próxima dele.

B) 9, 9 e 6 faltas; FIFO e LRU produzem o mesmo resultado nesta cadeia, e a escolha entre eles deve recair sobre o de menor custo de implementação.

C) 9, 8 e 7 faltas; o Ótimo não elimina as faltas de partida a frio e, por isso, fica apenas uma falta abaixo do LRU nesta simulação.

D) 10, 8 e 6 faltas; o FIFO penaliza as páginas mais antigas mesmo quando reutilizadas, gerando uma falta a mais que o LRU nas referências repetidas.

QUESTÃO 4 · GABARITO: A

A Simulando: FIFO 9 faltas (só o 0 da 5ª posição é acerto); LRU 8 (acertos nos 0 da 5ª e da 7ª); Ótimo 6 (poupa a página cujo próximo uso é mais distante: acertos em 0, 0, 2 e 3). O Ótimo é referência inatingível; o LRU é a melhor aproximação prática.

B 9, 9 e 6 — trata o LRU como se fosse FIFO. Na 6ª referência (página 3) o LRU expulsa a página 1, a menos usada, e não a 0; por isso ganha o acerto na 7ª. Quem 'empata' os dois não simulou.

C 9, 8 e 7 — Ótimo mal simulado. Na 6ª referência a vítima correta é a página 1, que nunca mais aparece; expulsar a 2 gera uma falta extra na 9ª. O Ótimo nunca perde para o LRU.

D 10, 8 e 6 — conta a 5ª referência (página 0, já residente) como falta do FIFO. Acerto não gera substituição: é erro de contagem, não de algoritmo.

Questão 4 · resolução passo a passo

A simulação, quadro a quadro

Cadeia de referências 7·0·1·2·0·3·0·4·2·3 · 3 quadros · memória inicialmente vazia

FIFO

7	0	1	2	0	3	0	4	2	3
7	7	7	**2**	2	2	2	**4**	4	4
	0	0	0	0	**3**	3	3	**2**	2
		1	1	1	1	**0**	0	0	**3**
F	F	F	F	•	F	F	F	F	F

9 faltas

LRU

7	0	1	2	0	3	0	4	2	3
7	7	7	**2**	2	2	2	**4**	4	4
	0	0	0	0	0	0	0	0	**3**
		1	1	1	**3**	3	3	**2**	2
F	F	F	F	•	F	•	F	F	F

8 faltas

ÓTIMO

7	0	1	2	0	3	0	4	2	3
7	7	7	**2**	2	2	2	2	2	2
	0	0	0	0	0	0	**4**	4	4
		1	1	1	**3**	3	3	3	3
F	F	F	F	•	F	•	F	•	•

6 faltas

negrito = página que entrou no quadro naquela falta · FIFO tira a mais antiga · LRU a menos recentemente usada · ÓTIMO a que será usada mais tarde (ou nunca)

Elaboração dos autores.

5

Sistemas de Arquivos e Entrada/Saída

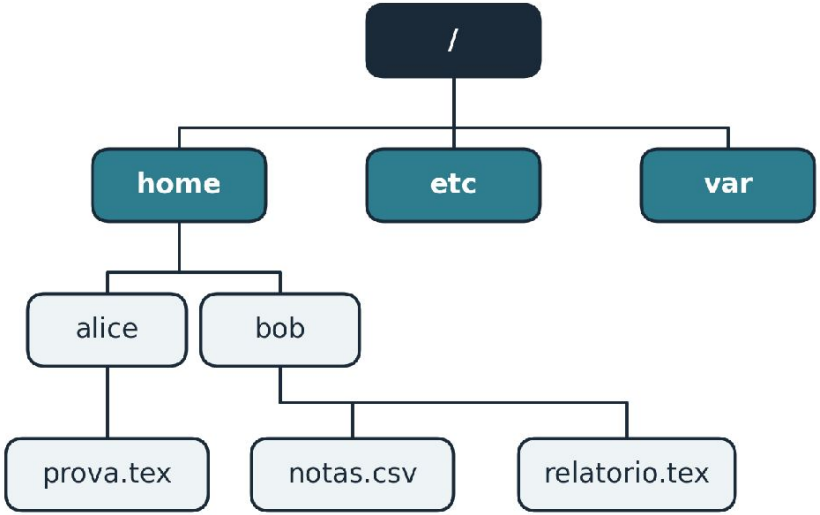
Tópicos 8 e 9 — organização em disco, diretórios, subsistema de E/S e proteção

T8 · Sistemas de arquivos: organização, diretórios e alocação em disco

T9 · Gerência de E/S, drivers, shell e compartilhamento de recursos

Arquivo, metadados e inode

DIRETÓRIO EM ÁRVORE



caminho absoluto: /home/alice/prova.tex

o NOME não fica no inode: fica na entrada do diretório

(hard links = vários nomes para o mesmo inode → contador de ligações)

O QUE FICA NO INODE

tipo e permissões
rw-r--r-- · arquivo comum, diretório, link

proprietário
UID e GID

tamanho
em bytes · contador de ligações (nº de nomes)

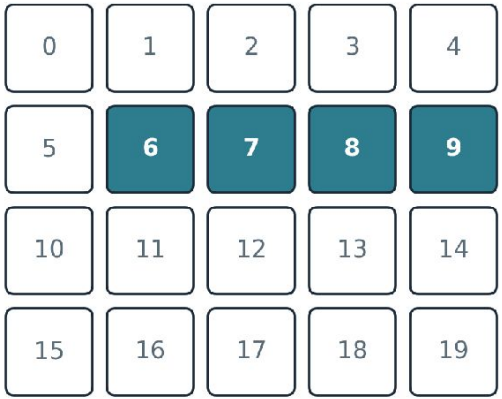
marcas de tempo
atime (último acesso) · mtime (conteúdo alterado)
ctime (metadados alterados — NÃO é 'criação')

ponteiros de blocos
diretos, indiretos simples, duplos e triplos

Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Alocação de arquivos em disco

CONTÍGUA

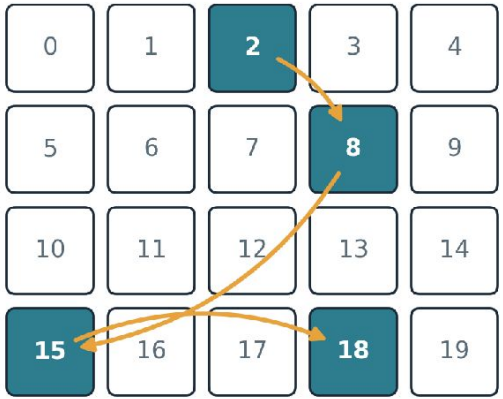


diretório: início = 6, tamanho = 4

bloco i do arquivo = 6 + i (acesso direto por cálculo)

ótima em sequencial · fragmentação externa · não cresce

ENCADEADA

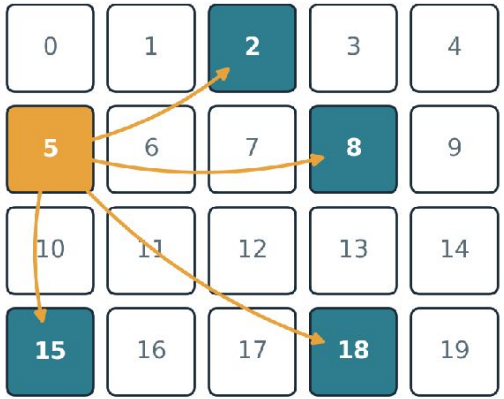


diretório: início = 2, fim = 18

cada bloco guarda o ponteiro do próximo: 2 → 8 → 15 → 18

sem frag. externa · acesso direto lento (percorre a cadeia)

INDEXADA



bloco de índice 5 → { 2, 8, 15, 18 }

acesso direto eficiente (uma consulta ao índice)

sem frag. externa · gasta espaço com o índice

Elaboração dos autores, com base em Tanenbaum e Bos (2016) e Silberschatz et al. (2018).

Espaços livres, journaling e sistemas de arquivos

Gerência de espaços livres — MAPA DE BITS (um bit por bloco — fácil achar sequências contíguas) × LISTA ENCADEADA (economiza espaço, mas não localiza contiguidade).

Journaling — grava as operações num diário ANTES de aplicá-las. Após falha, basta reaplicar ou descartar transações incompletas — dispensa varrer o volume inteiro.

ext4 e XFS — Linux, com journaling e inodes (alocação indexada multinível); XFS se destaca em arquivos muito grandes.

NTFS — Windows, com journaling, permissões por ACL e compressão.

FAT32 — alocação encadeada com tabela à parte; sem journaling e com limite de 4 GB por arquivo. Sobrevive pela compatibilidade universal.

E/S programada, por interrupção e por DMA

E/S programada (polling) — a CPU escreve o comando e fica em ESPERA OCUPADA, testando o registrador de status em laço, e move os dados ela mesma, byte a byte. Simples; desperdiça 100 % da CPU durante a transferência.

E/S por interrupção — a CPU dispara a operação e vai executar outro processo; o controlador interrompe a cada byte/palavra pronta e a CPU copia o dado. Libera a CPU entre eventos, mas uma interrupção por byte custa caro em dispositivos rápidos.

E/S por DMA — a CPU programa o controlador de DMA (endereço, contagem, sentido); ele transfere o bloco INTEIRO direto para a memória (roubo de ciclos do barramento) e gera UMA interrupção ao final. Menor sobrecarga — indispensável em disco e rede.

Camadas de E/S, buffering, caching, spooling e permissões

Camadas e drivers (Tanenbaum) — processo de usuário → software de E/S independente de dispositivo (nomeação, proteção, buffering, erros) → DRIVER (traduz o pedido genérico em comandos do controlador) → tratador de interrupção → hardware. O pedido desce; a resposta sobe.

Buffering e caching — buffer: área intermediária que absorve diferença de velocidade ou de tamanho de transferência. Cache: cópia de blocos recentes em memória para reuso (buffer cache de disco) — vive da localidade; write-back adia a gravação.

Spooling — fila em disco para um dispositivo DEDICADO (impressora): vários processos 'imprimem' ao mesmo tempo e um daemon envia um trabalho por vez. Transforma recurso exclusivo em compartilhável.

Shell e permissões no Linux — rwx para dono, grupo e outros (chmod 754 = rwxr-xr--); em diretório, r = listar e x = atravessar; redirecionamento (>, <, 2>) e pipe (|) ligam a saída de um comando à entrada do outro.

QUESTÃO 5 · ENADE

Questão de Afirmativa Negativa (CAT 1)

SITUAÇÃO-PROBLEMA

Ao revisar a configuração de um servidor que acumula as funções de arquivo e de impressão, um administrador redige as premissas que orientaram suas decisões sobre o subsistema de entrada e saída. A equipe de arquitetura precisa identificar premissas equivocadas antes de aprovar o documento.

COMANDO

Considerando o funcionamento do subsistema de entrada e saída e os mecanismos de proteção de recursos, assinale a alternativa que apresenta uma afirmação INCORRETA.

ALTERNATIVAS

A) Na E/S por acesso direto à memória (DMA), o controlador transfere blocos inteiros entre o dispositivo e a memória principal e interrompe o processador apenas ao final da transferência, reduzindo a sobrecarga da CPU em relação à E/S por interrupção.

B) O spooling desacopla a aplicação do dispositivo ao gravar a saída em uma área intermediária em disco, permitindo que vários processos submetam trabalhos simultaneamente enquanto um processo servidor os envia à impressora, um de cada vez.

C) O buffer cache mantém em memória principal blocos de disco recentemente acessados e aproveita o princípio da localidade de referência para reduzir o número de acessos físicos ao dispositivo.

D) Em sistemas Linux, as permissões de leitura, escrita e execução aplicadas ao proprietário, ao grupo e aos demais usuários são verificadas pelo núcleo, e o bit de execução em um diretório controla a possibilidade de atravessá-lo para alcançar seu conteúdo.

E) Na E/S programada por polling, o processador delega ao controlador o controle completo da transferência e permanece livre para executar outros processos, sendo por isso a técnica mais eficiente para dispositivos de alta taxa de transferência.

QUESTÃO 5 · GABARITO: E

A Essa é a definição precisa do DMA e de sua vantagem: uma única interrupção por bloco, em vez de uma por byte/palavra como na E/S por interrupção.

B Descreve fielmente o spooling, mecanismo que transforma um dispositivo não compartilhável, como a impressora, em recurso de uso concorrente por meio de uma fila em disco.

C É a descrição do buffer cache de disco e de seu fundamento — a localidade de referência —, o mesmo princípio que sustenta toda a hierarquia de memória.

D A verificação de permissões é atribuição do núcleo, e o bit x em diretórios de fato governa a travessia (possibilidade de acessar caminhos abaixo dele), distinta do bit r, que governa a listagem do conteúdo.

E A E/S programada por polling faz exatamente o oposto: o processador permanece em espera ocupada, testando repetidamente o registrador de status do controlador e movendo os dados ele mesmo, byte a byte.

Referências

TANENBAUM, A. S.; BOS, H. Sistemas operacionais modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Fundamentos de sistemas operacionais. 9. ed. Rio de Janeiro: LTC, 2018.

MACHADO, F. B.; MAIA, L. P. Arquitetura de sistemas operacionais. 5. ed. Rio de Janeiro: LTC, 2013.

STALLINGS, W. Arquitetura e organização de computadores. 10. ed. São Paulo: Pearson, 2017.

BRASIL. INEP. Exame Nacional de Desempenho dos Estudantes — provas de Ciência da Computação (bacharelado), edições de 2017 e 2021. Brasília: Inep.

Nota: todos os diagramas deste material são de elaboração dos autores, esquematizados com base nas obras acima; o modelo de PCB e os estados suspensos seguem Machado e Maia (2013).